

<https://www.youtube.com/watch?v=OYvlznJ4lZQ&t=1247s>

1 Large language model

A **large language model** is a type of AI model trained on vast amounts of text to **predict the next token** (word or piece of a word). By learning these patterns, it can generate and transform text, answer questions, summarize, translate, and write code.

How it works (high level)

- **Training:** Learns statistical patterns in text by minimizing next-token prediction error.
- **Architecture:** Most modern LLMs use the **Transformer** architecture with **attention**.
- **Inference:** Given a prompt, it generates tokens step by step using a decoding strategy (for example, sampling).

2 Tokenisation

Tokenisation is the process of splitting text into **tokens** (words, subwords, or characters) that the model can read.

- Example: "Tokenisation is fun" →
 - Tokens (approx.):
 - "Token"
 - "isation"
 - " is"
 - " fun"

3 Vectors

A **vector** is a list of numbers that represents something (like a word, sentence, image, or user).

- **Why use them:** numbers are easy for models to compare and learn from.
- **Meaning:** similar things get vectors that are close together in space.
- **Example (idea):** “cat” and “kitten” → vectors that are near each other.
- **Where you see them:**
 - **Embeddings:** vectors that store meaning.
 - **RAG search:** compare your question vector to document chunk vectors to find the best matches.
 - **Attention:** tokens are turned into vectors (queries, keys, values) to decide what to focus on.

4 Attention Mechanism

The **attention mechanism** lets a model decide **which parts of the input to focus on** when producing each output token.

- **Intuition:** for every new word it writes, the model “looks back” at the most relevant earlier words.
- **How it works:** it computes *attention weights* (scores) between the current token and all other tokens, then makes a weighted mix of their information.
- **Why it matters:** it captures long-range relationships (for example, matching pronouns to the right noun) and enables the Transformer to process context efficiently.
- **Self-attention:** tokens attend to other tokens in the same sequence; **multi-head attention** runs several attention patterns in parallel (for example, one head tracks syntax while another tracks coreference).

5 Inheritance structure and self supervised learning

Inheritance (in ML) means building a model on top of a pre-trained one so it “inherits” general language skills, and self-supervised learning means the model teaches itself

from raw text by hiding or predicting parts of the text (like the next word) without needing human-made labels.

6 Transformers

A **Transformer** is a neural network architecture that processes information in **parallel** using **attention**, which helps it model relationships between tokens (words or subwords) over long ranges.

The core idea

- Instead of reading tokens one by one (like many older RNN-based models), Transformers look at **all tokens at once**.
- For each token, the model computes how much it should “pay attention” to every other token.

Main components (high level)

- **Token + position information:** tokens are turned into vectors, then combined with **positional encodings** so the model knows order.
- **Self-attention:** each token mixes information from other tokens based on learned relevance scores.
- **Feed-forward network (FFN):** a small MLP applied to each token to add non-linearity and capacity.
- **Residual connections + LayerNorm:** stabilize training and help gradients flow.
- **Stacked layers:** these blocks are repeated many times to build deeper understanding.

Why Transformers matter

- Scale well with data and compute.
- Capture long-range dependencies effectively.
- Enable today’s strongest LLMs and many vision and multimodal models.

7 Fine tuning

Fine-tuning means taking an already trained AI model and training it a little more on *your* data so it learns a specific style or task.

- **Start:** a general model (knows many things)
- **Add:** examples you care about (like Q&A pairs or labeled texts)
- **Train a bit:** adjust the model's weights
- **Result:** the model performs better on that specific job

Example: fine-tune a support chatbot using past support tickets so it answers like your team.

When to use: you need consistent behavior or a special domain style. If you only need the model to use your documents, **RAG** is often simpler.

8 Few Shot prompt

Few-shot prompting is a way to guide an LLM by giving it a *small set of examples* in the prompt, so it can infer the pattern you want.

Why it works

- The model learns the format, tone, and rules from the examples.
- It is often faster than fine-tuning for simple, repeatable tasks.

Mini example

- **Instruction:** Classify sentiment as Positive, Neutral, or Negative.
- **Examples:**
 - Input: "I loved it." → Output: Positive
 - Input: "It was okay." → Output: Neutral
 - Input: "I hated it." → Output: Negative
- **New input:** Input: "Not bad, but not great." → Output:

9 RAG

RAG (Retrieval-Augmented Generation) is a way to make an LLM answer using **your documents** by combining two steps:

1. **Retrieve:** find the most relevant chunks of text (often via **embeddings + vector similarity**).
2. **Generate:** pass those chunks into the prompt so the model answers grounded in that context.

10 Vector Database

A vector database is a system that stores “meaning vectors” (number lists) for things like text and lets you quickly find the most similar items to your query by comparing those vectors.

11 Model Context Protocol

Model Context Protocol (MCP) is an open standard that lets AI apps connect to external tools and data sources (for example, files, databases, and APIs) in a consistent way.

- **Why it exists:** instead of every AI tool building custom integrations, MCP defines a shared interface.
- **How it works (conceptually):**
 - An AI app (the *client*) discovers available tools and resources from an MCP server.
 - The app can then call tools (for example, search, read, write) with structured inputs.
 - The server returns structured results the model can use in its responses.
- **What you get:** safer, more reliable “tool use”, easier integration maintenance, and better grounding in real data.

- **Common use cases:** connecting a chat assistant to company docs, issue trackers, calendars, or internal services.

12 Context Engineering

Context engineering is the practice of deliberately shaping what an AI model “sees” at inference time so it produces more accurate, consistent, and grounded outputs.

What you control

- **Instruction:** the goal, constraints, and success criteria.
- **Input:** the user’s query plus any relevant details.
- **Retrieved context (RAG):** the most relevant document chunks.
- **Tool context:** outputs from tools (search, database queries, APIs).
- **Conversation memory:** what must persist vs. what should be forgotten.
- **Output format:** schemas, templates, and examples.

Common techniques

- **Role + rules:** state the model’s job and hard constraints up front.
- **Few-shot examples:** show the desired pattern for predictable tasks.
- **Structured prompts:** headings like “Task / Context / Constraints / Output”.
- **Context selection:** include only what’s necessary; remove noise.
- **Chunking + citations:** pass smaller, high-signal excerpts and require citations.
- **Guardrails:** ask for clarification when context is missing or conflicting.

Quick checklist

- What decision is the model making?
- What facts must be grounded in provided sources?
- What should the output look like (format + length)?
- What must not happen (hallucinations, unsafe actions, speculation)?

13 Agents

An **agent** is an AI helper that can do multi-step work for you, often by using tools and information you connect (like your docs, calendar, or other apps).

- **You give a goal:** “Summarize this doc”, “Draft an email”, “Plan my week”.
- **It takes steps:** it can break the goal into smaller tasks.
- **It can use tools:** search your workspace, read pages, write drafts, or call connected apps.
- **It remembers context:** it uses the instructions and context you provide to stay on track.
- **It returns an output:** a summary, checklist, plan, or updated page.

Example

- Goal: “Create study notes from this page.”
- Steps: read the page → extract key ideas → make a clean outline → add examples.

When to use agents

- Repetitive work (summaries, formatting, drafting).
- Multi-step tasks (research → outline → write).
- Work that needs tool use (finding info across pages).

14 Reinforcement Learning

Reinforcement learning (RL) is a way to train an AI by letting it **learn through trial and error**.

- The AI (the *agent*) takes an action.
- The environment gives a **reward** (good) or **penalty** (bad).
- Over time, the agent learns a **policy**: what actions get the highest total reward.

Example: a game bot tries moves, gets points for winning, and learns which moves work best.

15 Chain of thoughts

Chain-of-thought is the step-by-step reasoning a model uses to reach an answer.

- **Why it helps:** it breaks a problem into small steps, so the final answer is usually more accurate.
- **In practice:** you often only see the final answer, but the model still “thinks” in steps internally.

16 Reasoning mode

Reasoning mode refers to a model setting that prioritizes careful, step-by-step problem solving over speed or creativity.

- **What changes:** the model spends more compute per answer, checks intermediate steps, and is less likely to guess.
- **Trade-off:** typically **slower** and may be **more expensive**, but can improve accuracy on complex tasks (math, logic, multi-step planning).
- **When to use:** high-stakes or multi-step questions where correctness matters more than latency.

17 Multimodal models

A **multimodal model** can understand and generate across **multiple data types** (modalities), not just text.

- **Common modalities:** text, images, audio, video, structured data.
- **Example tasks:** describe an image, answer questions about a chart, transcribe audio, generate an image from text.

- **Why it matters:** many real-world problems combine text + visuals or audio, so multimodal models can work with more realistic inputs.

18 Small language models (SLMs)

A **small language model (SLM)** is a language model with fewer parameters and lower compute requirements than large frontier LLMs.

- **Benefits:** cheaper to run, faster, easier to deploy on-device or on limited servers.
- **Trade-offs:** usually lower performance on broad, open-ended tasks, but can be strong when focused on a domain.
- **Common uses:** edge devices, privacy-sensitive environments, high-volume customer support, embedded assistants.

19 Distillation

Distillation is a technique for training a **smaller model** (student) to mimic a **larger model** (teacher).

- **How it works:** the student learns from the teacher's outputs (often probabilities or "soft targets"), not just ground-truth labels.
- **Why it helps:** keeps much of the teacher's behavior while reducing size and cost.
- **Typical outcome:** a faster, cheaper model with some accuracy loss, but often a good trade for production.

20 Quantization

Quantization makes models **smaller and faster** by storing weights and doing computations in **lower precision** (for example, 16-bit → 8-bit or 4-bit).

- **Benefits:** less memory, faster inference, lower hardware cost.
- **Trade-offs:** can reduce accuracy if pushed too far or done poorly.

-
- **Where it's used:** deploying models on GPUs/CPUs with limited memory, mobile or edge devices, or when serving many requests.